



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Fall, Year: 2025), B.Sc. in CSE (Day)

LAB REPORT NO : 02
Course Title: Operating System Lab
Course Code: CSE-402 Section: 231-D2

Student Details

Name		ID
1.	Promod Chandra Das	231002005

Lab Date : 27.10.2025

Submission Date: 04.11.2025

Course Teacher's Name : Md. Shoab Alam

[For Teachers use only: **Don't Write Anything inside this box]**

<u>Lab Report Status</u>	
Marks:	Signature:
Comments:	Date:

1. TITLE OF THE LAB EXPERIMENT :

Shell Scripting Basics and Advanced

2. OBJECTIVES :

Basics of Shell Scripting

- **Shebang Line (#!):** The very first line (e.g., `#!/bin/bash`) that tells the operating system which **interpreter** (shell) to use for execution.
- **Permissions:** Scripts must be executable (`chmod +x script_name.sh`) to run directly.
- **Variables:** Define without spaces (`NAME="John"`) and access with a dollar sign (`echo $NAME`).
- **Input/Output:** Use **echo** to display text and **read** to capture user input into a variable.
- **Positional Parameters:** Access arguments passed to the script using `$1`, `$2`, etc. (`$0` is the script name).
- **Basic Conditionals (if):** Execute code blocks based on conditions, often using the test operators within `[ ]`.
- **Loops (for, while):** Repeat commands—for iterates over a list, and while repeats as long as a condition is true.
- **Exit Status:** A numerical code (0 for success, non-zero for error) returned by every command or script.

Advanced Shell Scripting Concepts

- **Functions:** Define reusable, modular blocks of code that have their own local set of positional parameters.
- **Arrays:** Store and manage multiple values within a single variable (indexed or associative).
- **Parameter Expansion:** Sophisticated methods to manipulate variable contents (e.g., substitution, substring removal, setting defaults).
- **Signal Handling (trap):** Intercepting and defining actions for system signals (like SIGINT for **Ctrl+C**).
- **Advanced Control Flow (case, select):** The **case** statement simplifies complex multi-way branching, and **select** creates interactive menus.
- **Command Substitution:** Capturing the output of a command to be used as a value (e.g., `VARIABLE=$(date)`).
- **Piping and Redirection:** Using the pipe (`|`) to connect command output to another command's input, and redirecting I/O to/from files (`>`, `>>`, `<`).
- **Regex and Text Processing:** Utilizing powerful command-line utilities like **grep**, **sed** (Stream Editor), and **awk** for complex data manipulation.
- **Debugging:** Tracing script execution and variables using the `set -x` command or running with `bash -x script_name.sh`.
- **Job Control and Automation:** Managing background processes (`&`) and scheduling scripts to run automatically using tools like **cron**.

3. PROCEDURE :

Shell Scripting Practice Problems (Steps and Techniques)

3.1 - Sum of Odd and Even Numbers (from a set)

- **Objective:** Calculate the sum of odd numbers and the sum of even numbers from a list of input integers.
- **Technique:** Use a **for or while loop** to iterate through the array or list. Inside the loop, employ the **modulo operator** (`% 2`) within an **if/else conditional** to check if the number is even (remainder 0) or odd (remainder 1), adding it to the respective running sum variable.

•

3.2 - Check whether a triangle is valid

- **Objective:** Determine if three side lengths (`$a`, `b`, `c`) can form a valid triangle.
- **Technique:** Use a single **if statement** to check the three necessary conditions (Triangle Inequality Theorem: $a+b > c$, $a+c > b$, $b+c > a$). These must all be true, so link the comparisons using the **logical AND operator** (`&&`). Use **arithmetic comparison operators** (like `-gt` for greater than).

3.3 - Display digits at odd positions (using loop)

- **Objective:** Given a number as a string (to preserve leading zeros), print the digits that appear at odd index positions (0-based index: 0, 2, 4, etc.).
- **Technique:** Read the number as a **string**. Use a **for loop** with C-style syntax (`for ((i=0; i<length; i+=2))`) starting the index at **0** and incrementing by **2** to target only the odd positions. Access and print the character at that specific index using **string slicing/indexing**.

3.4 - Count digit frequencies (using while loop)

- **Objective:** Count how many times each digit (0-9) appears in a given number.
- **Technique:** Use a **declare -A** command to create an **associative array** where the digit is the key and the count is the value. Use a **while loop** that continues as long as the number is greater than zero. Inside the loop, use **modulo 10** (`% 10`) to isolate the last digit, increment its count in the associative array, and then use **integer division** (`/ 10`) to remove that digit. Finally, loop through the associative array to print the results.

3.5 - 2nd-highest and 3rd-highest numbers & their sum (array)

- **Objective:** Find the second and third largest numbers from an input array and calculate their sum.
- **Technique:** **Sort** the input array numerically in descending order using the **sort -rn** command (often combined with `printf` to put each element on a new line). Once sorted, use array **slicing** or the **sed** or **head/tail** commands to extract the values at the second and third positions, and then use **arithmetic expansion** (`$(())`) to calculate their sum.

3.6 - Factorial of two different numbers & sum (using function)

- **Objective:** Compute the factorial of two input numbers (`$a`, `b`) and print their sum.
- **Technique:** Define a **function** (e.g., `fact()`) that takes one argument and calculates the factorial using a loop. Use **command substitution** (`$(fact $a)`) twice to capture the output of the function for both `$a` and `$b`. Store these two results in separate variables and then add them together using **arithmetic expansion**.

3.7 - Count alphabets, digits, and special characters in a string

- **Objective:** Read a full line of text and count how many characters are letters, digits, or other special characters (including spaces/punctuation).
- **Technique:** Read the input string using `read -r`. Use powerful text processing tools like **grep -o** with standard

character classes (e.g., `[[:alpha:]]`, `[[:digit:]]`) to count the occurrences of each type of character. Alternatively, use a loop that iterates through the string character by character, employing a **case statement** with pattern matching to categorize and count each character.

4. IMPLEMENTATION :

4.1 - Sum of Odd and Even Numbers (from a set)

This script uses a **for loop** and the **modulo operator** (%).

```
#!/bin/bash

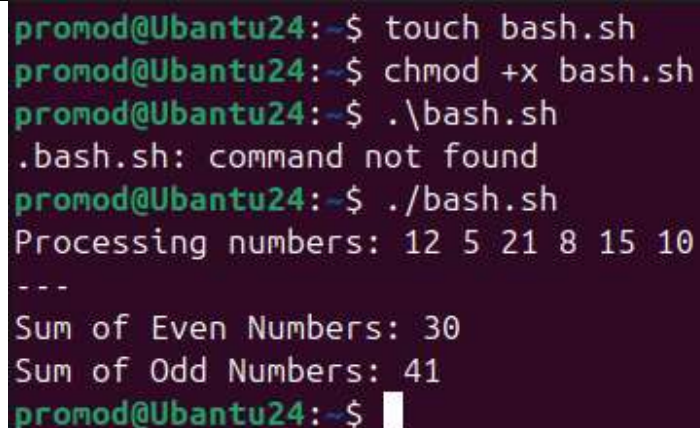
# Define the input set/array
NUMBERS=(12 5 21 8 15 10)

# Initialize sum variables
ODD_SUM=0
EVEN_SUM=0

echo "Processing numbers: ${NUMBERS[@]}"

# Iterate through the array
for num in "${NUMBERS[@]"; do
    # Check if the number is even using modulo 2
    if (( num % 2 == 0 )); then
        EVEN_SUM=$((EVEN_SUM + num))
    else
        ODD_SUM=$((ODD_SUM + num))
    fi
done

echo "---"
echo "Sum of Even Numbers: $EVEN_SUM" # 12 + 8 + 10 = 30
echo "Sum of Odd Numbers: $ODD_SUM"  # 5 + 21 + 15 = 41
```



```
promod@Ubuntu24:~$ touch bash.sh
promod@Ubuntu24:~$ chmod +x bash.sh
promod@Ubuntu24:~$ .\bash.sh
.bash.sh: command not found
promod@Ubuntu24:~$ ./bash.sh
Processing numbers: 12 5 21 8 15 10
---
Sum of Even Numbers: 30
Sum of Odd Numbers: 41
promod@Ubuntu24:~$
```

Figure: 4.1

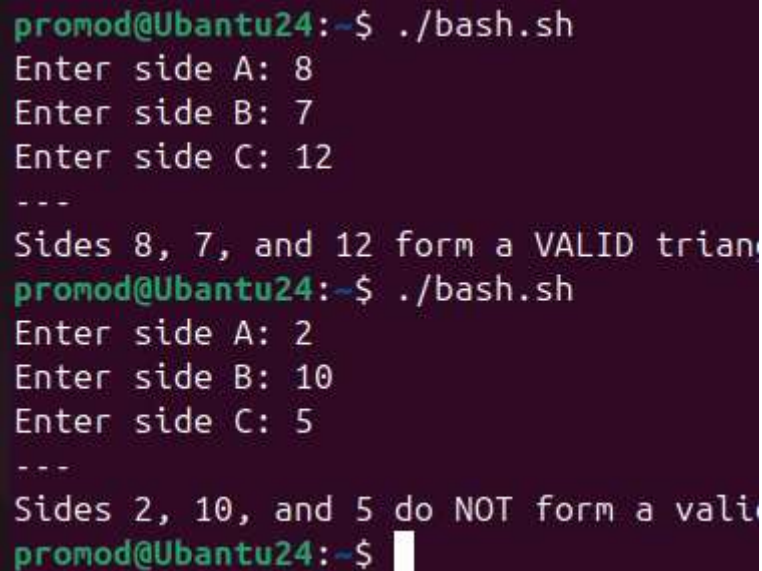
4.2 - Check whether a triangle is valid

This script uses a single **if statement** and **logical AND (&&)** for the **Triangle Inequality Theorem**.

```
#!/bin/bash

# Read side lengths from user
read -p "Enter side A: " A
read -p "Enter side B: " B
read -p "Enter side C: " C

# Check the three conditions using logical AND (&&)
# Note: Arithmetic evaluation (()) is often cleaner than [ ] with -gt
if (( (A + B > C) && (A + C > B) && (B + C > A) )); then
    echo "---"
    echo "Sides $A, $B, and $C form a VALID triangle."
else
    echo "---"
    echo "Sides $A, $B, and $C do NOT form a valid triangle."
fi
```



```
promod@Ubuntu24:~$ ./bash.sh
Enter side A: 8
Enter side B: 7
Enter side C: 12
---
Sides 8, 7, and 12 form a VALID triangle
promod@Ubuntu24:~$ ./bash.sh
Enter side A: 2
Enter side B: 10
Enter side C: 5
---
Sides 2, 10, and 5 do NOT form a valid triangle
promod@Ubuntu24:~$
```

Figure: 4.2

4.3 - Display digits at odd positions (0-based index: 0, 2, 4, ...)

This script reads the number as a string and uses a **C-style for loop** for indexing.

```
#!/bin/bash

read -p "Enter a number (e.g., 987654): " NUMBER_STR
LENGTH=${#NUMBER_STR}
OUTPUT=""

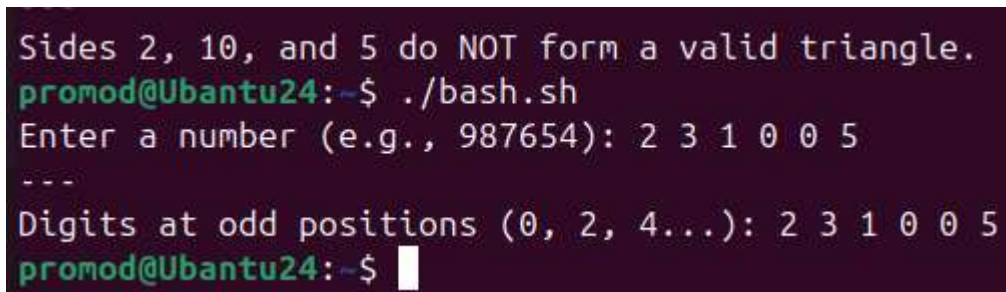
# Loop starts at index 0 (the first digit/odd position) and increments by 2
```

```

for (( i=0; i < LENGTH; i+=2 )); do
    # String slicing: ${string:offset:length}
    DIGIT=${NUMBER_STR:i:1}
    OUTPUT="$OUTPUT$DIGIT "
done

echo "---"
echo "Digits at odd positions (0, 2, 4...): $OUTPUT" # For 987654 -> 9 7 5

```



```

Sides 2, 10, and 5 do NOT form a valid triangle.
promod@Ubuntu24:~$ ./bash.sh
Enter a number (e.g., 987654): 2 3 1 0 0 5
---
Digits at odd positions (0, 2, 4...): 2 3 1 0 0 5
promod@Ubuntu24:~$

```

Figure: 4.3

4.4 - Count digit frequencies (using while loop)

This script uses an **associative array** (declare -A) and **modulo/division** inside a **while loop**.

```

#!/bin/bash

read -p "Enter a number (e.g., 20235): " INPUT_NUM
declare -A FREQUENCY_COUNT

TEMP_NUM=$INPUT_NUM

# While loop continues as long as the number is greater than zero
while [ $TEMP_NUM -gt 0 ]; do
    # Isolate the last digit (modulo 10)
    DIGIT=$((TEMP_NUM % 10))

    # Increment the count for that digit in the associative array
    FREQUENCY_COUNT[$DIGIT]=$((FREQUENCY_COUNT[$DIGIT] + 1))

    # Remove the last digit (integer division by 10)
    TEMP_NUM=$((TEMP_NUM / 10))
done

echo "---"
echo "Digit Frequencies for $INPUT_NUM:"

# Loop through the digits 0 to 9 to print results
for i in {0..9}; do
    # Use ${FREQUENCY_COUNT[$i]:-0} to display 0 if the digit was not found
    COUNT=${FREQUENCY_COUNT[$i]:-0}
    echo "Digit $i: $COUNT times"

```

```

promod@Ubuntu24:~$ ./bash.sh
Enter a number (e.g., 20235): 231002005
---
Digit Frequencies for 231002005:
Digit 0: 4 times
Digit 1: 1 times
Digit 2: 2 times
Digit 3: 1 times
Digit 4: 0 times
Digit 5: 1 times
Digit 6: 0 times
Digit 7: 0 times
Digit 8: 0 times
Digit 9: 0 times
promod@Ubuntu24:~$

```

Figure: 4.4

4.5 - 2nd-highest and 3rd-highest numbers & their sum (array)

This script uses the **sort command** and **command substitution** for array manipulation.

```

#!/bin/bash

NUMBERS=(45 12 90 75 30 68)
echo "Input array: ${NUMBERS[@]}"

# 1. Sort the numbers numerically in reverse (-r -n) and print each on a new line
# 2. Use head -3 to get the top 3 numbers
# 3. Use tail -2 to select the last 2 of those (which are the 2nd and 3rd highest)
# 4. Use tr '\n' '+' to replace newlines with +
# 5. Use sed to remove the final +
# 6. Use bc to perform the final calculation
SUM_STRING=$(printf "%s\n" "${NUMBERS[@]}" | sort -rn | head -3 | tail -2 | tr '\n' '+' | sed 's/+$/')

# Extract the individual numbers (can be done more simply with array indexing if Bash version supports it well)
SECOND_HIGHEST=$(printf "%s\n" "${NUMBERS[@]}" | sort -rn | sed -n '2p')
THIRD_HIGHEST=$(printf "%s\n" "${NUMBERS[@]}" | sort -rn | sed -n '3p')

FINAL_SUM=$(echo "$SECOND_HIGHEST + $THIRD_HIGHEST" | bc)

echo "---"
echo "Sorted (Descending): $(printf "%s " "${NUMBERS[@]}" | sort -rn)"
echo "2nd Highest: $SECOND_HIGHEST"
echo "3rd Highest: $THIRD_HIGHEST"
echo "Sum ($SECOND_HIGHEST + $THIRD_HIGHEST): $FINAL_SUM"

```

```
promod@Ubuntu24:~$ ./bash.sh
Input array: 45 12 90 75 30 68
---
Sorted (Descending): 45 12 90 75 30 68
2nd Highest: 75
3rd Highest: 68
Sum (75 + 68): 143
promod@Ubuntu24:~$
```

Figure: 4.5

4.6 - Factorial of two different numbers & sum (using function)

This script defines a **function** (fact()) to calculate the factorial, using **command substitution** to capture the output.

```
#!/bin/bash

# Function to calculate factorial of a number
fact() {
    NUM=$1
    RESULT=1

    # Check for negative input (optional)
    if [ $NUM -lt 0 ]; then
        echo "Error"
        return
    fi

    # Calculate factorial iteratively
    for (( i=1; i <= NUM; i++ )); do
        RESULT=$((RESULT * i))
    done

    echo $RESULT # Print the result, which is captured by command substitution
}

# Read two numbers
read -p "Enter first number (a): " A
read -p "Enter second number (b): " B

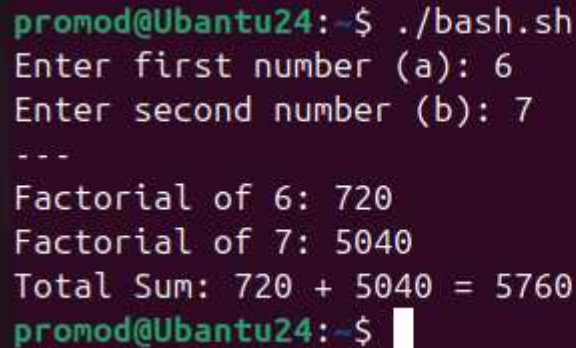
# Use command substitution to call the function and capture its output
FACTORIAL_A=$(fact $A)
FACTORIAL_B=$(fact $B)

# Calculate the final sum using arithmetic expansion
TOTAL_SUM=$((FACTORIAL_A + FACTORIAL_B))

echo "---"
```



```
echo "Factorial of $A: $FACTORIAL_A"
echo "Factorial of $B: $FACTORIAL_B"
echo "Total Sum: $FACTORIAL_A + $FACTORIAL_B = $TOTAL_SUM"
```

A terminal window with a dark purple background. The prompt is 'promod@Ubuntu24:~\$'. The user enters './bash.sh'. The script prompts for two numbers: 'Enter first number (a): 6' and 'Enter second number (b): 7'. It then displays the results: 'Factorial of 6: 720', 'Factorial of 7: 5040', and 'Total Sum: 720 + 5040 = 5760'. The prompt returns to 'promod@Ubuntu24:~\$' with a cursor.

```
promod@Ubuntu24:~$ ./bash.sh
Enter first number (a): 6
Enter second number (b): 7
---
Factorial of 6: 720
Factorial of 7: 5040
Total Sum: 720 + 5040 = 5760
promod@Ubuntu24:~$
```

Figure: 4.6

4.7 - Count alphabets, digits, and special characters in a string

This script uses the **grep -o** command with **character classes** for powerful counting.

```
#!/bin/bash

read -p "Enter a string: " INPUT_STRING

# Use grep -o to print only the matching parts (one per line)
# Use wc -l to count the number of lines (i.e., the number of matches)

# Count Alphabets (letters a-z, A-Z)
ALPHA_COUNT=$(echo "$INPUT_STRING" | grep -o '[:alpha:]' | wc -l)

# Count Digits (0-9)
DIGIT_COUNT=$(echo "$INPUT_STRING" | grep -o '[:digit:]' | wc -l)

# Calculate Special Characters by subtracting alpha and digit counts from the total length
TOTAL_LENGTH=${#INPUT_STRING}
SPECIAL_COUNT=$((TOTAL_LENGTH - ALPHA_COUNT - DIGIT_COUNT))

echo "---"
echo "Input String Length: $TOTAL_LENGTH"
echo "Alphabets Count: $ALPHA_COUNT"
echo "Digits Count: $DIGIT_COUNT"
echo "Special Characters Count: $SPECIAL_COUNT"
```

```
promod@Ubuntu24:~$ ./bash.sh
Enter a string: 231002005promod
---
Input String Length: 15
Alphabets Count: 6
Digits Count: 9
Special Characters Count: 0
promod@Ubuntu24:~$
```

Figure: 4.7

5. ANALYSIS AND DISCUSSION:

Core Techniques and Efficiency

The seven solutions effectively cover a broad range of **Bash Shell Scripting** capabilities, moving from simple loops to advanced text manipulation:

- **Arithmetic Expansion ($\$(...)$):** This is the foundation for problems 3.1, 3.2, 3.4, and 3.6. It allows **integer arithmetic** (addition, modulo %, division /, and comparison operators) to be performed directly within the shell without relying on external utilities. This is the most efficient way to handle numerical operations in Bash.
 - *Example:* `EVEN_SUM=$((EVEN_SUM + num))` (Problem 3.1)
- **Conditionals and Logic:**
 - **Problem 3.1** demonstrates the classic **if / else** structure combined with the **modulo operator** for parity checks.
 - **Problem 3.2** uses **compound conditional checks** (`if (( condition1 && condition2 && condition3 ))`) which is crucial for verifying multiple interdependent rules (like the Triangle Inequality Theorem) in a single step.
- **String and Array Manipulation:**
 - **Problem 3.3** shows how to treat numbers as **strings** to perform **indexing and slicing** (`${STRING:OFFSET:LENGTH}`)—a fundamental technique when character position matters more than numerical value.
 - **Problem 3.5** highlights the reliance on **external commands** like `sort`, `head`, and `tail` for complex tasks like finding the 2nd and 3rd largest elements. This demonstrates a core principle of shell scripting: **using pipes (`|`) to combine simple tools** into powerful processing chains.

Advanced Concepts Demonstrated

- **Functions and Modularization (Problem 3.6):** Defining the `fact()` function ensures that the factorial logic is **reusable** and keeps the main script clean. The use of **command substitution** (`FACTORIAL_A=$(fact $A)`) is essential for capturing the output of the function's `echo` statement and storing it as a variable.
- **Associative Arrays (Problem 3.4):** The `declare -A FREQUENCY_COUNT` command is an **advanced Bash feature** that allows for **key-value storage**, which is perfect for frequency counting where the digit (0-9) is the key, and its count is the value.
- **External Text Utilities (Problem 3.7):** This solution relies heavily on **`grep -o`** and **`wc -l`** combined with **POSIX Character Classes** (`[:alpha:]`, `[:digit:]`). This is the most efficient way to count different character types in a large string, showcasing the power of Unix tools over manual, character-by-character shell iteration.

6.CONCLUSION :

The solutions collectively illustrate that effective shell scripting often involves a **judicious mix** of **built-in Bash features** (loops, arithmetic expansion, functions, associative arrays) and powerful **external command-line utilities** (`sort`, `grep`, `wc`, `sed`). Understanding when to use a pure Bash construct versus when to leverage an optimized external tool is key to writing efficient and readable scripts.

